Problem Set 2

Quan Wen

Due 3 November 2025

NOTE: This problem set requires coding.

1 Rall's equation for an infinite long cable

We introduced to you the cable equation that describes the propagation of the signal in a dendrite. It reads

$$c_m \frac{\partial V}{\partial t} = \frac{a}{2\rho_L} \frac{\partial^2 V}{\partial x^2} - i_a(x,t) - i_e(x,t), \tag{1}$$

where c_m is the specific membrane conductance per unit area, i_a as the membrane conductance per unit area, and i_e is the external current per unit area.

In a linear approximation of the membrane current, we have

$$i_a = g_m(V - E_L),$$

where E_L is the resting potential of the neuron, and $g_m = 1/r_m$ is a constant membrane conductance per unit area, and r_m is specific membrane resistance. Now by introducing a new variable $v = V - E_L$, the cable equation can be simplified as

$$c_m r_m \frac{\partial v}{\partial t} = \frac{a r_m}{2\rho_L} \frac{\partial^2 v}{\partial x^2} - v - i_e(x,t) r_m.$$

Note that r_m has the dimension $\Omega{\cdot}\text{mm}^2$, and ρ_L has the dimension $\Omega{\cdot}\text{mm}$, and thus we could define a critical spatial scale λ, called electrotonic length,

$$\lambda = \sqrt{\frac{a r_m}{2\rho_L}} \tag{2}$$

Moreover, recall the membrane time constant

$$\tau_m = c_m r_m. \tag{3}$$

We could rewrite the cable equation as

$$\tau_m \frac{\partial v}{\partial t} = \lambda^2 \frac{\partial^2 v}{\partial x^2} - v - i_e(x,t) r_m. \tag{4}$$

Solve a special case for 4 for an infinitely long cable by injecting a point current at point $x = 0$ that does not change over time. Find the steady state solution that is independent of time, that is,

$$\lambda^2 \frac{d^2 v}{dx^2} = v + i_e r_m$$

where

$$i_e = -\frac{I_e}{2\pi a}\delta(x)$$
$$v|_{x=\infty} = 0$$
$$v|_{x=-\infty} = 0$$

2 The Perceptron Learning Algorithm

In class, we discussed the Perceptron learning algorithm, which is guaranteed to find a decision plane to separate two linearly-separable groups of points. In this exercise, you will write a Matlab code to implement the algorithm and use it to analyze numerically the perceptron capacity by using the threshold activation function studied in class.

The perceptron update rule changes the decision plane by cycling through the P given data-points one at a time and rotating the plane slightly when misclassifying a sample. The rule can be formulated as follows at time t.

$$\text{If } \left(w^{(t)} \cdot x^{\mu}\right) y_0^{\mu} < 0 \rightarrow w^{(t+1)} = w^{(t)} + \eta y_0^{\mu} x^{\mu}. \tag{5}$$

where $x^{\mu}, w \in \mathbb{R}^N$, $\mu = 1 \ldots P$, and $y_0^{\mu} = -1, 1$. The weights vector w can start from any position and η (the learning rate parameter) can be any value and this algorithm will converge to a solution in a finite number of steps (assuming the data is indeed linearly separable).

2.1 Implement in MATLAB function

$$\text{function } [w, \text{converged}, \text{epochs}] = \text{perceptron}(x, y0)$$

The function receives a matrix X (of size $N \times P$) of the samples and a vector y_0 (of size $P \times 1$) of the corresponding labels. The function should return the final weight vector w (of size $N \times 1$), a flag indicating if the algorithm converged and the number of epochs. In each epoch, we go over a cycle of P different input samples. Some of the inputs will be misclassified and the weight vectors will be updated according the learning rule specified by Eq. (1). The algorithm converges if no inputs are misclassified in the last P input samples. The total number of epochs is just given by the total number of the input samples that have been checked during iterations divided by P. Set in your function a maximum number of epochs, which once exceeded the function terminates with a non-convergence indication. In the next sections you will use the algorithm

to explore the convergence abilities and perceptron capacity. You'll do it by generating random binary inputs $x_i^\mu \in [-1, 1]$ and outputs. Run your algorithm and check convergence. Repeat each combination of N, P several times to collect statistics (e.g., $n = 50$).

2.2 Compare number of epochs

Compare number of epochs till convergence for $P = 10$ and $N = 10$, 20, 100. How do the changes in N affect the convergence speed? Explain.

2.3 Convergence probability

Define $\alpha = P/N$. Plot graphs of convergence probability as a function of $0 < \alpha < 3$. Discuss your results and compare them to results of Cover's function counting theorem that were derived in class.

2.4 Margin analysis

One of the methods to grade different hyperplanes that correctly separate the training data, is to check their distance from the training points. We'll expect hyperplanes with a higher distance to have a smaller generalization error. The margin δ of a hyperplane is defined as the minimal distance from it to the set of training points: $\delta^* = \min_\mu[(w \cdot x^\mu)y_0^\mu]$. Calculate the mean margin for different (P, N) combinations you used to evaluate convergence probability in **1.3** (naturally, you'll use the statistics from converged runs). Use the mean margin to calculate a margin histogram $H(P, N)$. Use the histogram to discuss - how do changes in P, N affect the hyperplane margin?

2.5 Sparse Coding

If an output of '+1' represents a spike, we might want to check the capacity of the perceptron at different firing rates regimes. We do this by introducing some bias in the patterns. Generate input/output patterns in which the probability of $y_0^\mu = +1$ is f (and also the probability of $x_i^\mu = +1$ is f). Explore the capacity at $N = 100$ and for several values of f. Discuss your results.

2.6 Excitatory Synapses

Next, we will modify our algorithm to allow only 'excitatory synapses', i.e., $w_i \geq 0$. After each update step clip all 'negative synapses' to zero. In addition, we add a constant non-negative threshold θ for the perceptron, i.e., $y(x^\mu) = \text{sign}(w \cdot x^\mu - \theta)$. Compare the capacity of this modified perceptron to the original one (use unbiased input/output patterns). You will have to experiment with your learning parameter η, to find a regime in which the algorithm converges in a satisfactory manner (try: $\eta = 10^{-1}, 10^{-2}, \ldots, 10^{-5}$).